Building an LLM from scratch

a tiny GPT that learns to write like Shakespeare, one character at a time

818,241 parameters • 1,115,394 characters of training text • device: cpu
(ChatGPT-class models have ~1,000,000x more parameters. Same idea, much bigger.)

Step 1 · The data

An LLM learns from raw text. We use Tiny Shakespeare: a plain .txt file with plays by Shakespeare.

First lines of the file:

```
First Citizen:
Before we proceed any further, hear me speak.

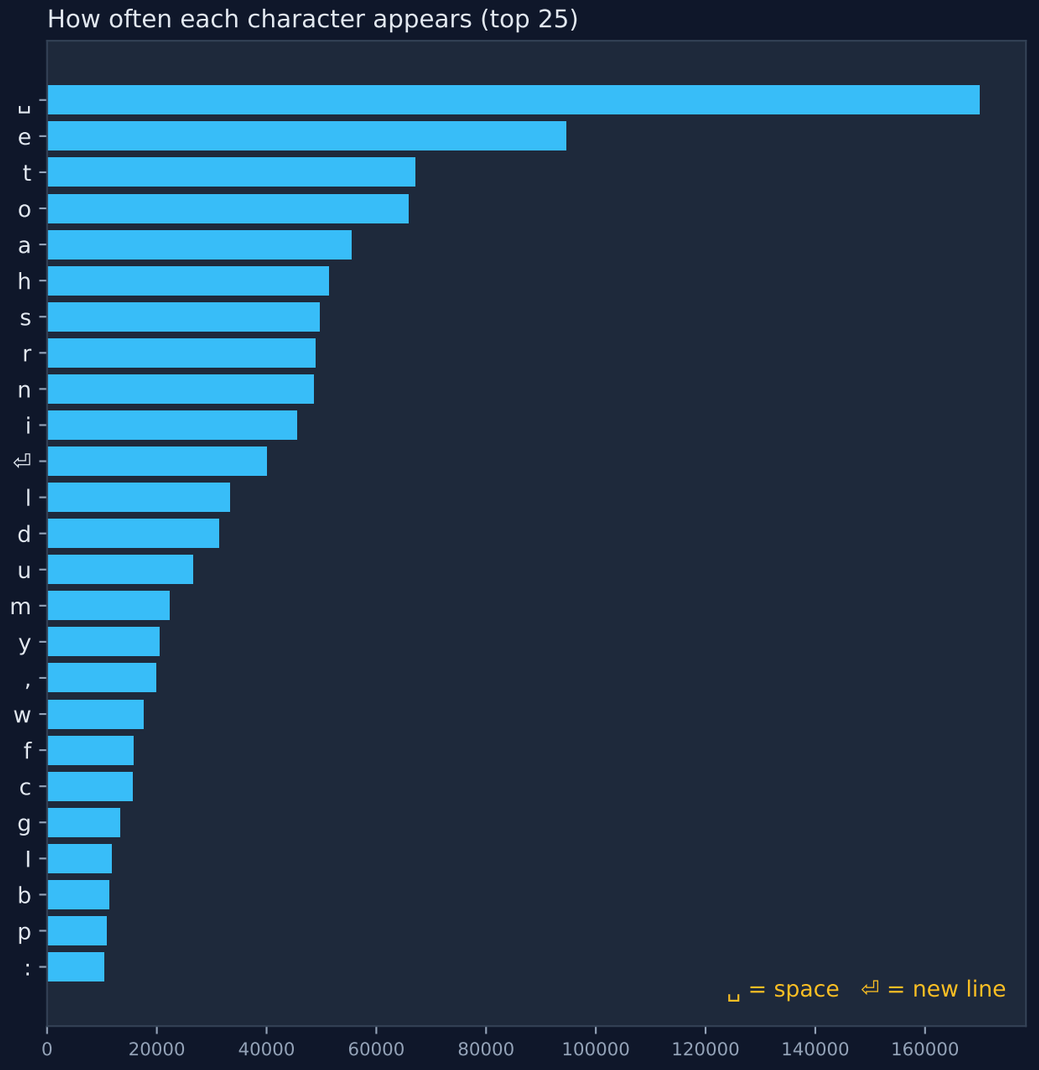
All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

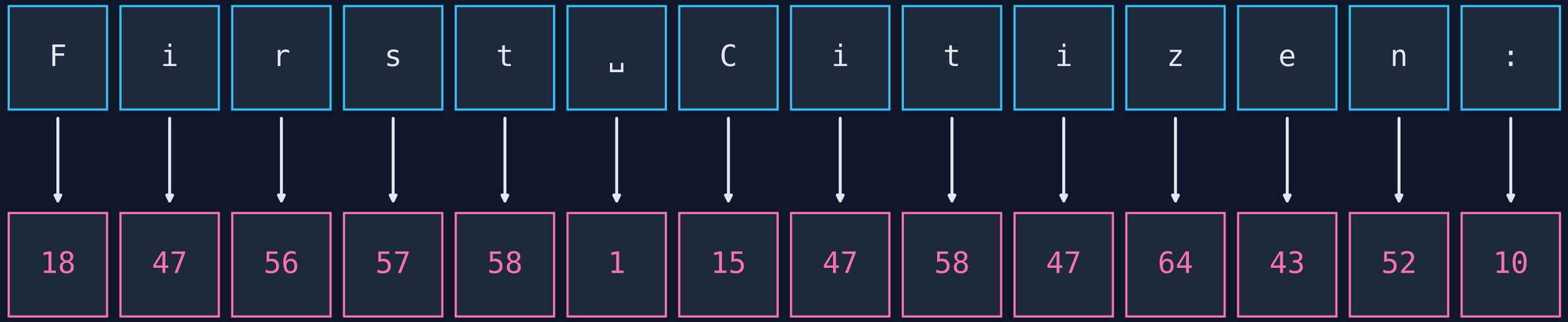
All:
We know't, we know't.
```



1,115,394 characters in total
65 different characters → our "vocabulary"
90% = 1,003,854 chars to learn from
10% = 111,540 chars kept hidden to test it

Step 2 · Tokenization: text → numbers

Neural networks only understand numbers. Every character in our vocabulary gets an ID. (ChatGPT uses chunks of words instead of single characters, but it's the same idea.)

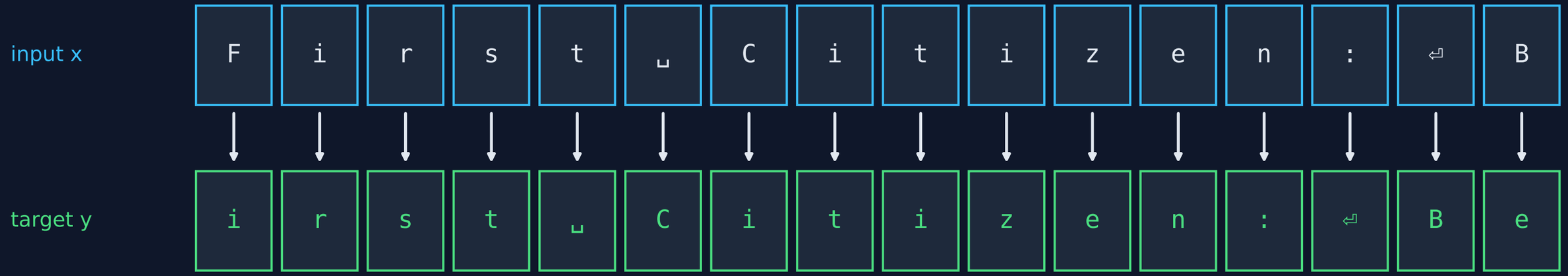


The complete vocabulary (id:character):

```
0:⏎ 1:␣ 2:! 3:$ 4:& 5:' 6:, 7:- 8:. 9:3 10:: 11:; 12:?
13:A 14:B 15:C 16:D 17:E 18:F 19:G 20:H 21:I 22:J 23:K 24:L 25:M
26:N 27:O 28:P 29:Q 30:R 31:S 32:T 33:U 34:V 35:W 36:X 37:Y 38:Z
39:a 40:b 41:c 42:d 43:e 44:f 45:g 46:h 47:i 48:j 49:k 50:l 51:m
52:n 53:o 54:p 55:q 56:r 57:s 58:t 59:u 60:v 61:w 62:x 63:y 64:z
```

Step 3 · Making training examples

The task: given the characters so far, predict the NEXT one. The answer key is simply the same text shifted by one character, so we never need to label anything by hand!



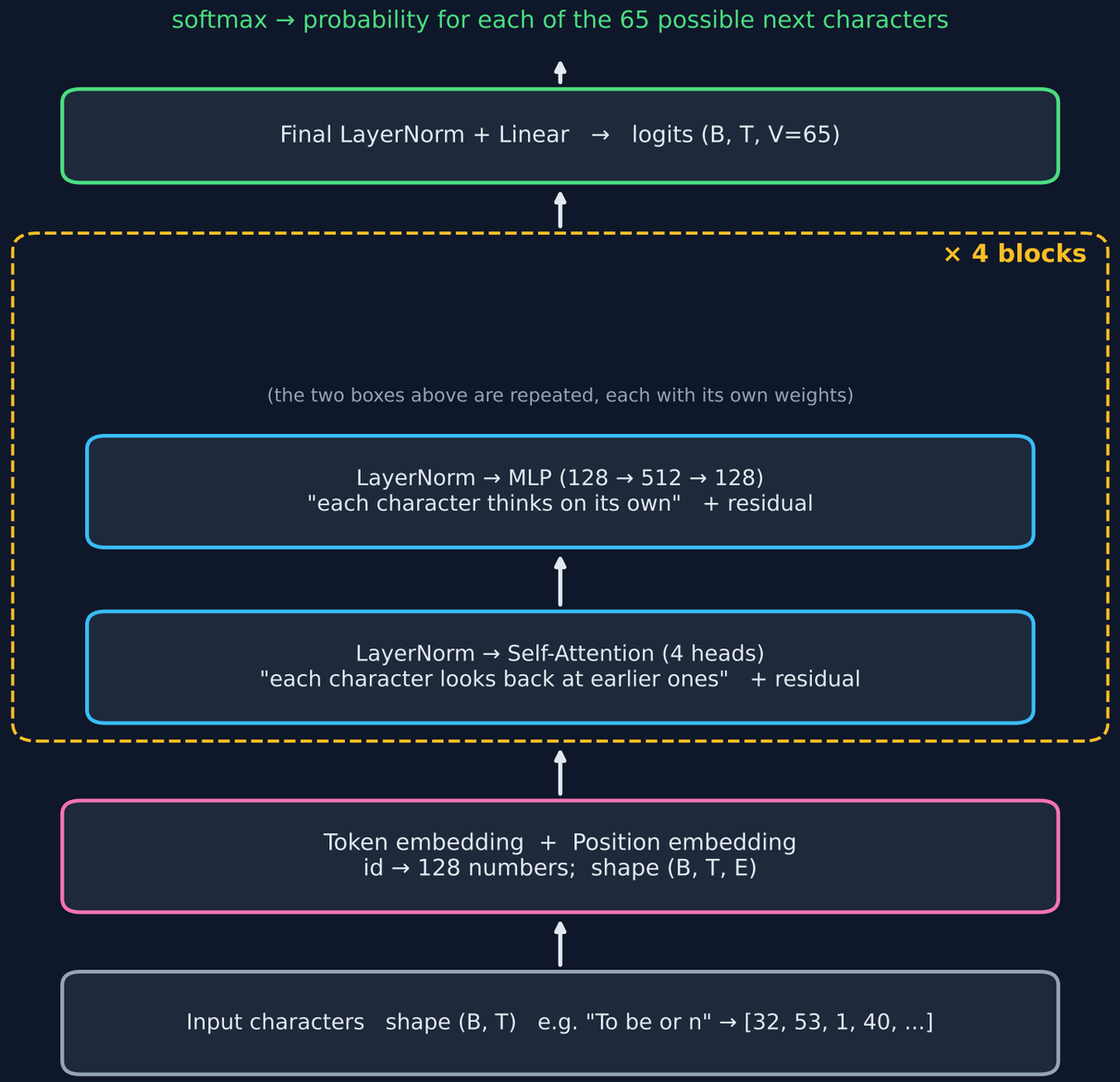
One chunk of 16 characters gives 16 separate lessons:

```
sees  'F'              →  must predict  'i'
sees  'Fi'             →  must predict  'r'
sees  'Fir'            →  must predict  's'
sees  'Firs'           →  must predict  't'
sees  'First'          →  must predict  ' '
sees  'First '         →  must predict  'C'
...
```

In training we grab 64 random chunks of 64 characters at once (a "batch")
= 64 x 64 = 4,096 predictions to check per training step.

Step 4 · The model: a Transformer (GPT)

Data flows bottom → top. B = batch size, T = number of characters, E = 128 (vector size), V = 65 (vocabulary).



Where the learnable numbers live

Token embedding	8,320	(65 x 128 lookup table)
Position embedding	8,192	(64 x 128)
Attention (per block)	66,048	
MLP (per block)	131,712	
Output layer	8,385	
TOTAL	818,241	<- these get nudged

These numbers are the model's "memory".

Before training they are small random values,
so the model knows nothing.

Training = slowly adjusting all of them so that
the correct next character gets a higher probability.

Before training: the model knows nothing

Random weights → it assigns almost the same probability to all 65 characters.

What it writes now (prompt: "ROMEO:"):

```
ROMEO:
KfCfec;mw;G!N-lOYVV&lPWmJTL !uJyd$NKEwp;IBYDC'E'SYJnlibmV
G&;$mp-ZCjcBCUU?oSl-IOlV?HfKdHsiv3NbUXuy:gatuFNPcrxcW?xjgBpFD,QHxbc, o BY$C
qI-y3;xWctZpUGwE?XijbQb$oite
lY$DAxDjwEyZwOZ,;Tq:PdlFi;&3y3H'VlUgIZwtAs-q;JIYvXD&

Z,wX.cVwyapi'R&f
OfT.hp
OGiV3yVhwxc&rgJC$CYvNcH
R;jVFXW qVq.OVAV.lGbBbEwNF,NMCO. dAgNI&bDryMfbpOVDCcxcehnFWg?I!Qj
```

Loss measures "how surprised is the model by the right answer": loss = -ln(P(correct char))

A pure random guess gives -ln(1/65) = 4.174

Our untrained model measures 4.171 ← almost exactly the random-guess number

Goal of training: push the loss down.

Step 5 · The training loop

These five steps repeat 3,000 times. Each repeat is called a "step". Next slide: watch it live.

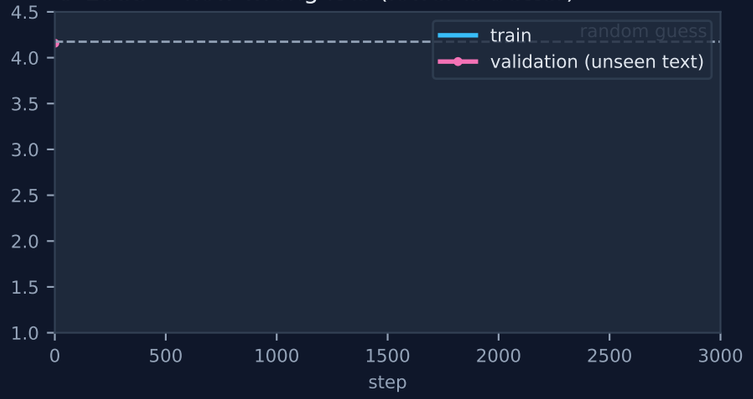


Optimizer: AdamW, learning rate 0.001 (warm-up, then slowly decreasing).
After many repeats: the right characters get higher probability → the loss falls → the text gets better.

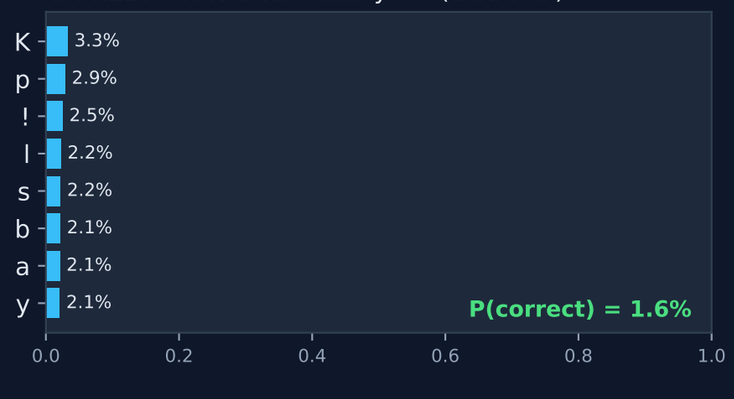
Step 5 · Training, live

Still almost random guessing (random = 4.17). It starts by learning which characters are common.

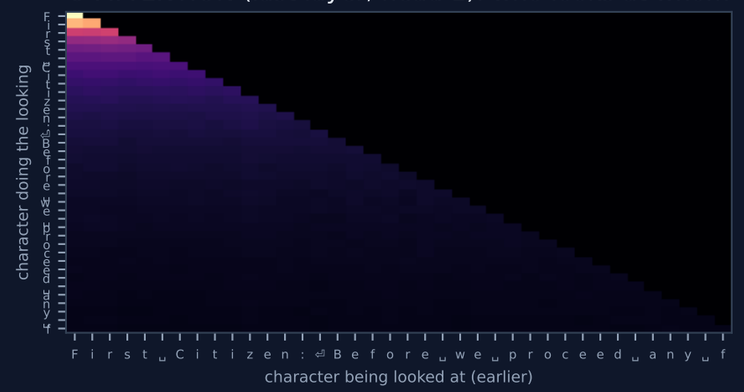
① LOSS = how wrong it is (lower is better)



② PREDICTION after: "...any f" (truth: 'u')



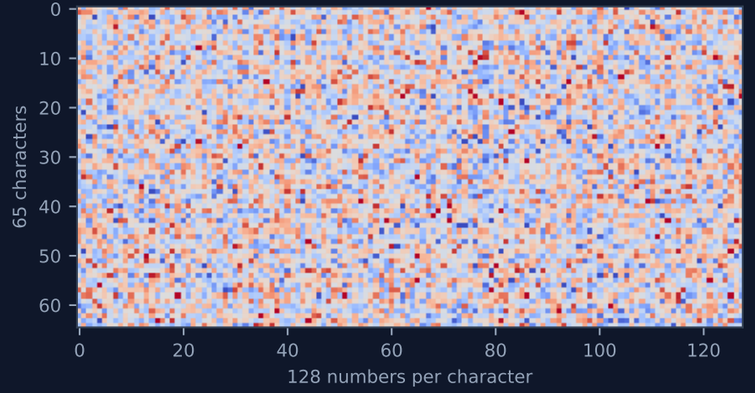
③ ATTENTION (last layer, head 1): who looks at whom



④ WHAT IT WRITES RIGHT NOW (prompt: "ROMEO:")

```
ROMEO:
odrLS' ckyGsghdHlRThjsynP$Sh
H..fuws:P;UJCfe$
zL$.ynfycgtawPy,Jy$Xx.BtIyWIvAsHNRAFXlI'bloFhbsKrgozwpTwcKEsIvZcmn
IpGZpmPUIFRAZQfAAyN'gXVPe
qUw.JiDKRDjVzlptGkl
.EbV?acjR
wVY:pi
uTd,JSJFDVlSnWjijmUs&f:hAVPlqRA,ydT3o?XkJ!l'QeR'kxyCQvooJKuSDztZDRFK?lP;J&:BjV
YSC;hw:tIB,$$G?ynJG,p&,&pbhsESFoegu Sn3pAUiGuH!aMmD
lXGFC!:k3v
ds
```

⑤ EMBEDDING TABLE colour = value, scale ±0.05



step	0 / 3,000
train loss	4.176
val loss	4.158
learning rate	0.00001
gradient size	3.820
chars seen	0
time / ETA	3s / 10421s

How to read this screen

① the line should go DOWN: the model is making fewer mistakes

② bars = its belief about the next letter; green bar = the true answer

③ bright square = this character is paying attention to that earlier one

④ free-running text from the current weights

⑤ every cell is a learnable number; each step nudges them a tiny bit

Step 5 · Training, live

Learning letter frequencies: lots of e, t, a, spaces and line breaks. Words aren't real yet.

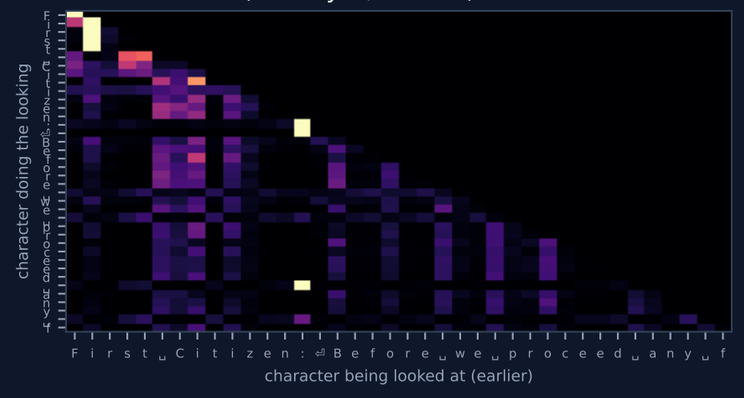
① LOSS = how wrong it is (lower is better)



② PREDICTION after: "...any f" (truth: 'u')



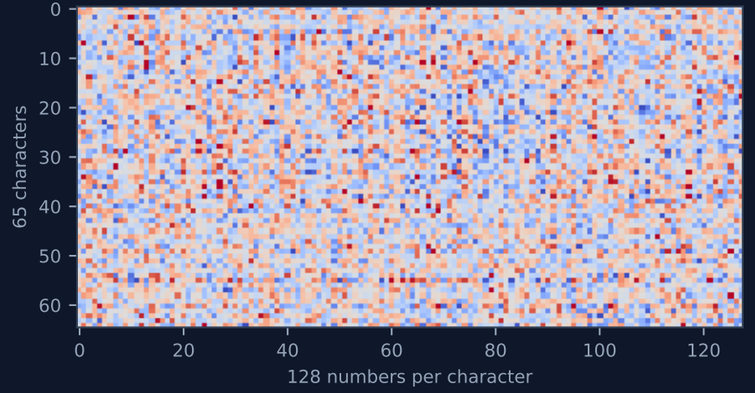
③ ATTENTION (last layer, head 1): who looks at whom



④ WHAT IT WRITES RIGHT NOW (prompt: "ROMEO:")

```
ROMEO:
odrLS' ckyGsghdHlRThjsynP$Sh
H..fuws:P;UJCfe$
zL$.ynfycgtawPy,Jy$Xx.BtIyWIvAsHNRAFXlI'bloFhbsKrgozwpTwcKEsIvZcmn
IpGZpmPUIFRAZQfAAyN'gXVPe
qUw.JiDKRDjVzlptGkl
.EbV?acjR
wVY:pi
uTd,JSJFDVlSnWjijmUs&f:hAVPlqRA,ydT3o?XkJ!l'QeR'kxyCQvooJKuSDztZDRFK?lP;J&:BjV
YSC;hw:tIB,$$G?ynJG,p&,&pbhsESFoegu Sn3pAUiGuH!aMmD
lXGFC!:k3v
ds
```

⑤ EMBEDDING TABLE colour = value, scale ±0.08



step	200 / 3,000
train loss	3.114
val loss	2.277
learning rate	0.00099
gradient size	0.787
chars seen	819,200
time / ETA	110s / 1536s

How to read this screen

① the line should go DOWN: the model is making fewer mistakes
② bars = its belief about the next letter; green bar = the true answer
③ bright square = this character is paying attention to that earlier one
④ free-running text from the current weights
⑤ every cell is a learnable number; each step nudges them a tiny bit

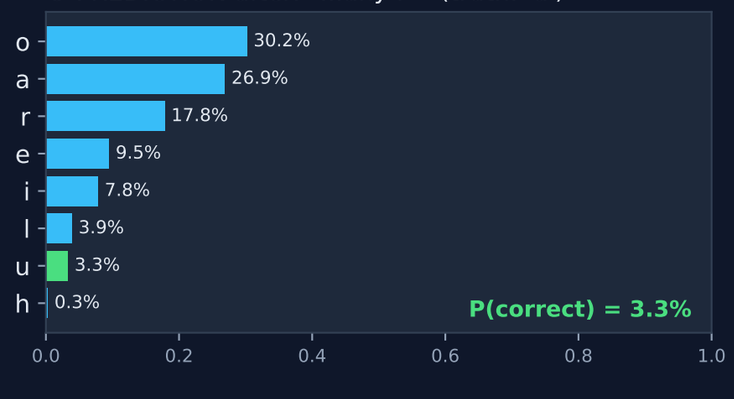
Step 5 · Training, live

It learns the script FORMAT: NAME in capitals, a colon, then a line of speech.

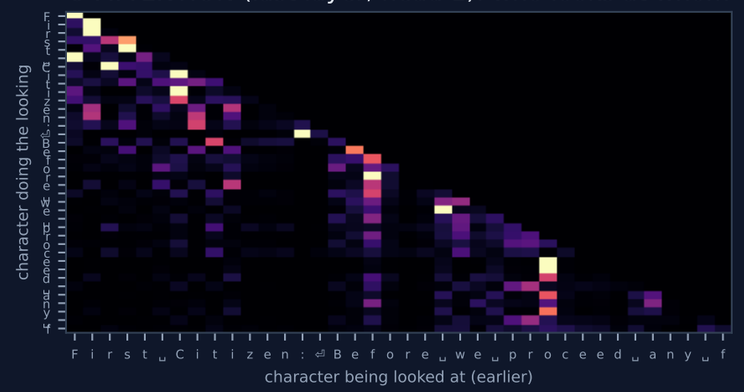
① LOSS = how wrong it is (lower is better)



② PREDICTION after: "...any f" (truth: 'u')



③ ATTENTION (last layer, head 1): who looks at whom



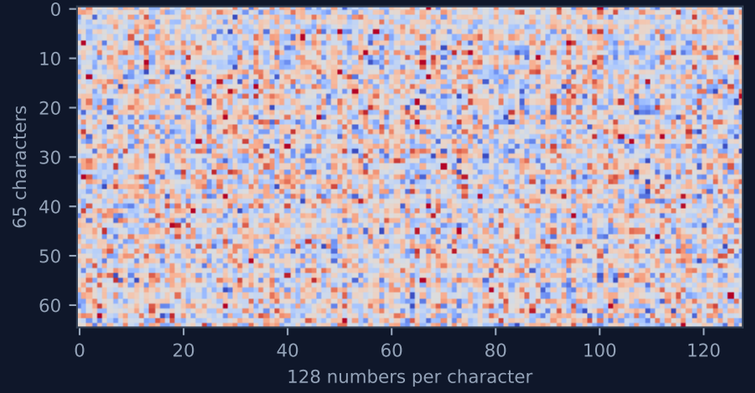
④ WHAT IT WRITES RIGHT NOW (prompt: "ROMEO:")

```
ROMEO:
I had I am blamblood well one of a that are
And take the so loveress the gate to were to well;
A mean I thoughting speaks I am of the lodd,
The is bloble, and it for he call rewell of York.

KING RIVERLINE:
Was of a fine him in some and eye earth day your to the life
To execome poor heir the glate!

SICIXIUS:
Capince!
```

⑤ EMBEDDING TABLE colour = value, scale ±0.13



step	1,000 / 3,000
train loss	2.051
val loss	1.778
learning rate	0.00078
gradient size	0.868
chars seen	4,096,000
time / ETA	529s / 1058s

How to read this screen

① the line should go DOWN: the model is making fewer mistakes
② bars = its belief about the next letter; green bar = the true answer
③ bright square = this character is paying attention to that earlier one
④ free-running text from the current weights
⑤ every cell is a learnable number; each step nudges them a tiny bit

Step 5 · Training, live

Word-like text with grammar-ish rhythm. About the best a model this tiny can do.

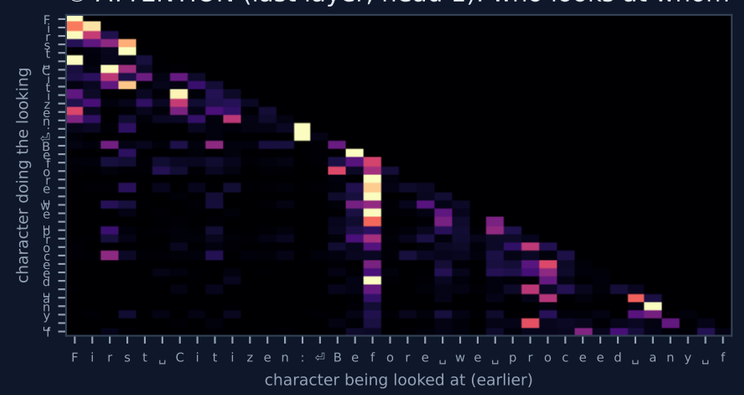
① LOSS = how wrong it is (lower is better)



② PREDICTION after: "...any f" (truth: 'u')



③ ATTENTION (last layer, head 1): who looks at whom



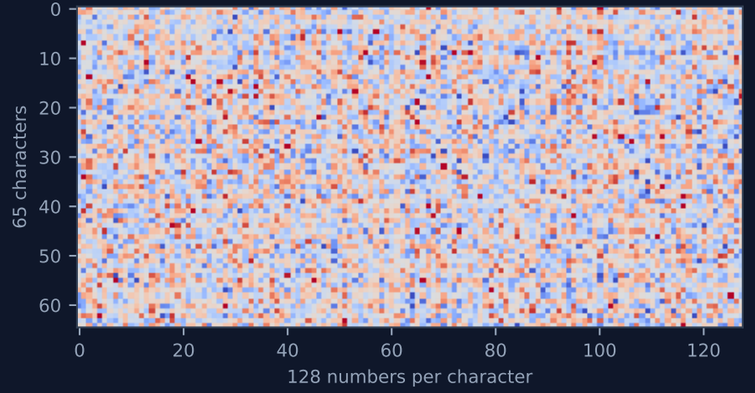
④ WHAT IT WRITES RIGHT NOW (prompt: "ROMEO:")

```
ROMEO:
Have I say because not prisons and the citizens
With the wont this heaven soverful for I heart,
Than as better with thy sons stranger soles,
Which he dost I will me in the wind.

PERDITA:
Let me as a the desires of the death;
The heavy is alanger my lord, and let Edward.

QUEEN MARGARET:
When they was where are satisfy
```

⑤ EMBEDDING TABLE colour = value, scale ±0.16



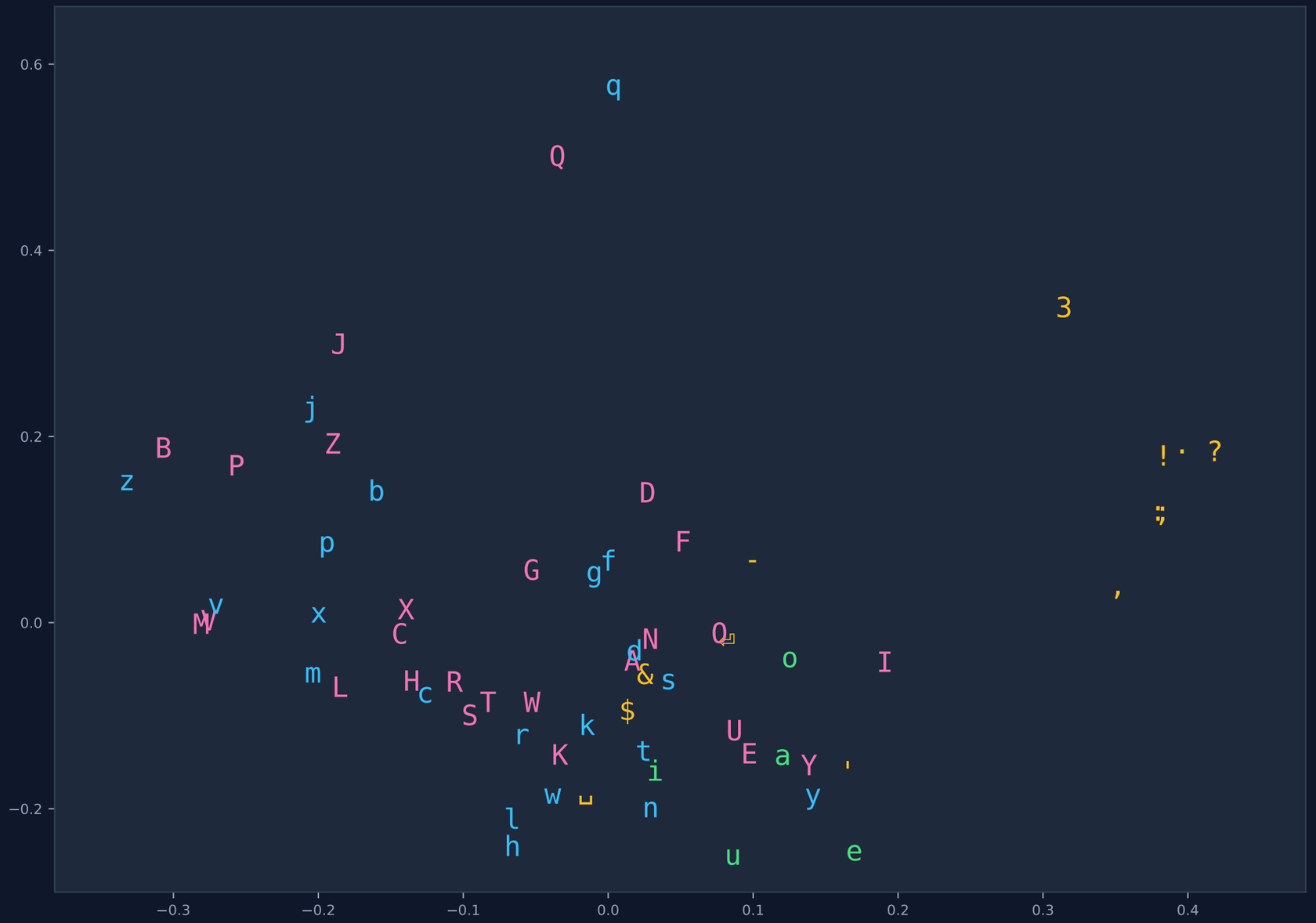
step	3,000 / 3,000
train loss	1.488
val loss	1.569
learning rate	0.00010
gradient size	0.852
chars seen	12,288,000
time / ETA	1556s / 0s

How to read this screen

① the line should go DOWN: the model is making fewer mistakes
② bars = its belief about the next letter; green bar = the true answer
③ bright square = this character is paying attention to that earlier one
④ free-running text from the current weights
⑤ every cell is a learnable number; each step nudges them a tiny bit

What it learned: similar characters end up close together

Each character is 128 numbers. Squashed to 2D here. Nobody told the model what a vowel is; it discovered the groups just by predicting the next letter.



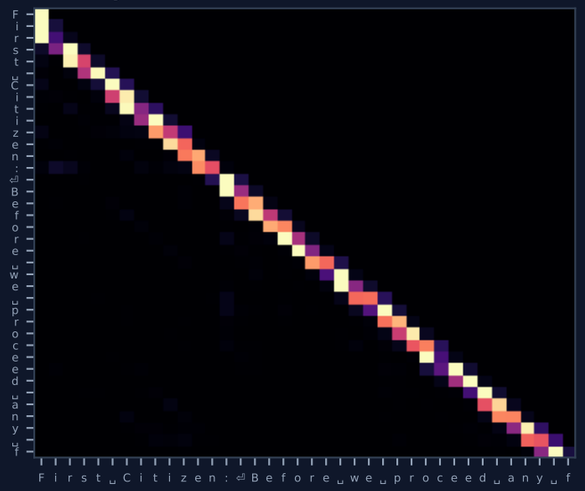
What it learned: attention heads

Each of the 4 heads in a layer learns its own way of looking back. Rows = the character looking, columns = where it looks. Bright = strong attention.

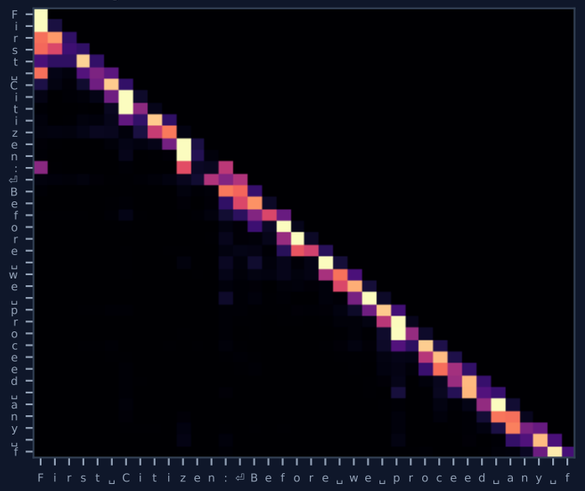
first layer, head 1



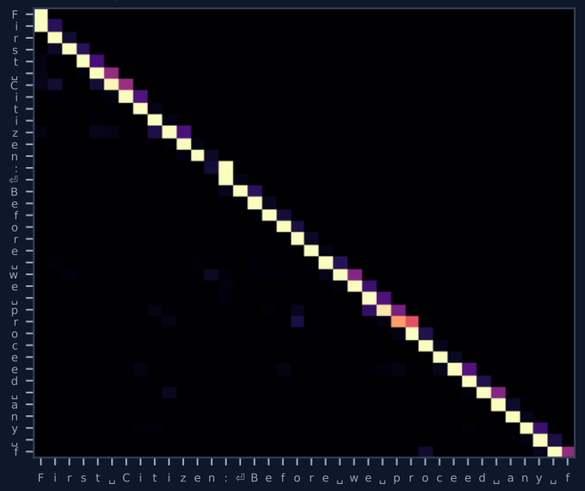
first layer, head 2



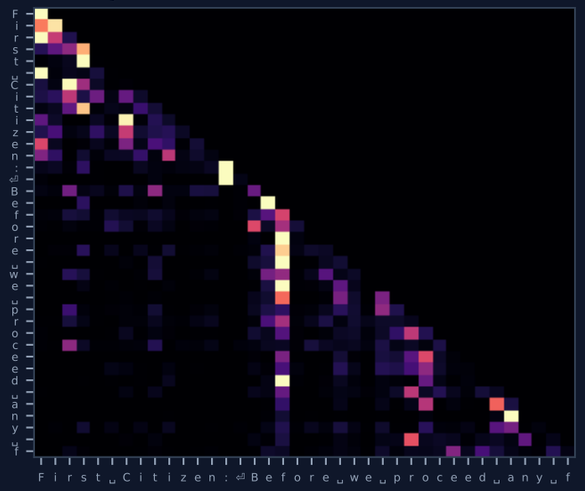
first layer, head 3



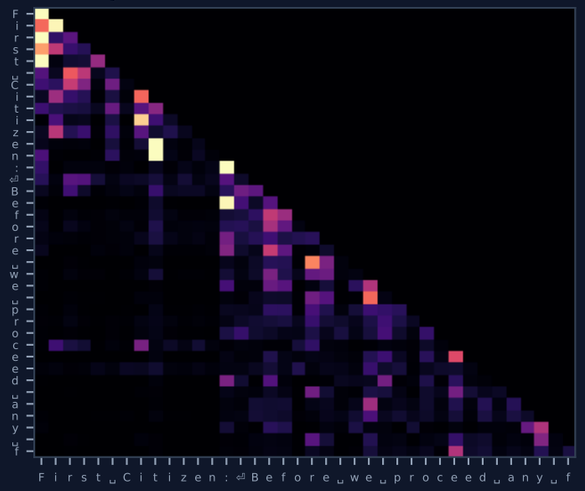
first layer, head 4



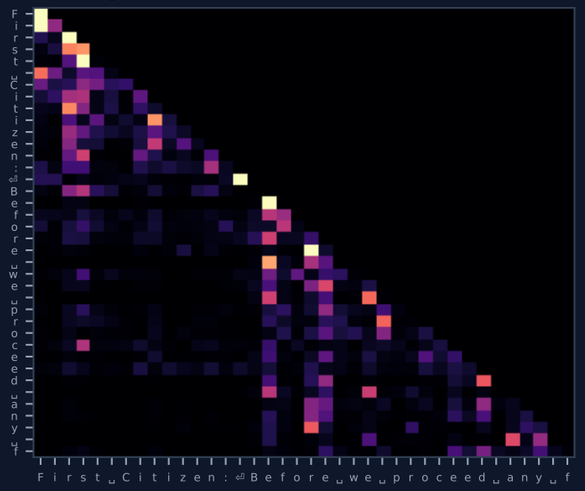
last layer, head 1



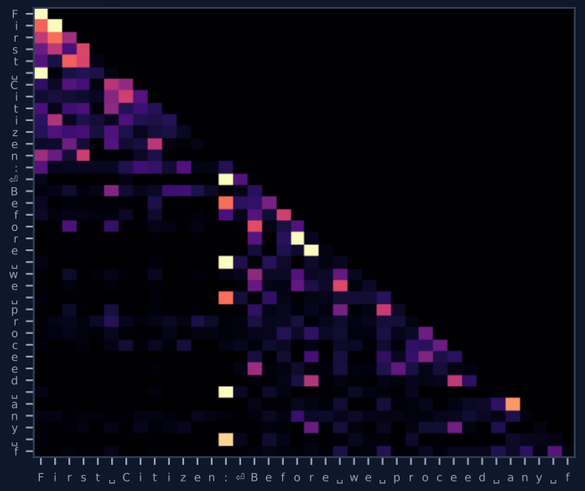
last layer, head 2



last layer, head 3



last layer, head 4



Generation: the temperature knob

The model outputs probabilities. Temperature decides how adventurous the sampling is. Low = safe and repetitive, high = creative but messy.

temperature = 0.5

```
ROMEO:
We shall not be so part to my death,
To this change the vow of the house
of state.

BRUTUS:
I will not our highness and my
heart,
Whose is of the point of our
countryment,
As if this the man, his metter sons
and shall are in my fa
```

temperature = 0.8

```
ROMEO:
Then, to your eyes, where hath the
first did to thy again;
And that I be time, for the time
about of stand.

AUTOLYS:
I must be a treasure time, tell it
it with than
He do away no frest for his mother's
shake to the na the
rest wh
```

temperature = 1.2

```
ROMEO:
Pray his duke-sour crows in and ruke
see;,
For he is shall I widam like twe
fevost:
Train do shortwixt his bone, your
lady one own:
Turning sir, as I not dog stiff, fey
now let redment,
Is unto the husbard, Edward's
metitinger as
```

Now YOU talk to it

IMPORTANT: this is NOT ChatGPT. It was never taught to answer questions, only to continue text.
So you don't ask it things, you START a piece of text and it keeps writing.

```
Prompts to try (type \n for a new line):

  ROMEO:\n                        → writes a speech in Romeo's name
  KING HENRY:\nMy lord,            → continues in royal style
  First Citizen:\nWe are           → street-crowd dialogue
  To be, or not to be               → mangles a famous line
  JULIET:\nO Romeo, Romeo!         → a dramatic answer
  hello how are you                 → chaos: it never saw modern English
```

What to expect:

- Mostly real English words, a few invented ones ("thou dost bewitch'd")
- Correct play format: CAPITAL NAME + colon + lines of speech
- Sentences that sound Shakespearean but make no real sense
- Plenty of mistakes: it has only 1 MB of text and ~0.8M parameters

Chat commands: /temp 0.5 /len 500 /quit